

TraveloAPP Partner API

Partner integration manual

The interface a partner uses to search departures, reserve and confirm tickets under its own name, and cancel them when needed.

VERSION 1.05 · ISSUED 30 Aug 2026

Before you start

You receive the **base URL** — separate for test and production — together with your credentials. Every path in this manual is appended to it:

```
<base-url>/auth/api_sales_login
<base-url>/search_trip
...
```

Moving from test to production changes only the address; request bodies and the control code calculation stay the same.

What you get from us:

Item	Purpose
TID	your terminal identifier
OTP	password used to log in
k	secret key for the control code — never sent in any request

The key `k` never travels over the network. It is used to **sign** requests, so keep it on your server; anyone holding it can order in your name.

Rate limits: login 10 requests per minute, all other calls 120 per minute per IP address.

Format: Content-Type: application/json . Dates are YYYY-MM-DD .

Login

```
POST /auth/api_sales_login
Content-Type: application/json

{ "tid": "SAB00K001", "otp": "<your-otp>" }
```

```
{ "msg": "token created", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I.."} }
```

The token is valid for **24 hours** and goes into the header of every further request:

```
Authorization: Bearer <token>
```

An expired or invalid token returns `401`. Log in again — do not log in before every call.

Control code

Requests that move money or create an obligation are signed with the `control_code` field. The code is **SHA512, hexadecimal**, computed over the key `k` and the field values **in exactly this order**:

Endpoint	What is concatenated
<code>/search_trip</code>	<code>k + travel_from + travel_date + travel_to</code>
<code>/order</code>	<code>k + order_number + number of items + sum of total_item_price</code>
<code>/confirm_order</code>	<code>k + order_uuid + order_number + total_amount</code>
<code>/cancel_order</code>	<code>k + order_uuid + order_number + total_amount</code>

Numbers are concatenated as printed (`2` , `50`), with no spaces or separators.

For confirmation and cancellation you **do not send** `order_number` and `total_amount` — we take them from the order on our side, while you put them into the code from the response you received when the order was created. That way the code cannot be built over an amount that is not ours.

Example (Node.js):

```
const crypto = require("crypto");

const control_code = crypto
  .createHash("sha512")
  .update(k + "HR479" + "2026-09-05" + "HR364")
  .digest("hex");
```

If the code does not match, the response is `400 Invalid control_code`.

Sales flow

An order goes through three states:

```
/order      -> DRAFT      seats reserved, tickets do not exist yet
/confirm_order -> CONFIRMED tickets issued, seats consumed
```

```
/cancel_order    ->  CANCELED    tickets voided, seats released
```

An unconfirmed order binds neither you nor us, but it **holds the seats** — do not leave it open longer than your payment takes.

1. Harbors

```
GET /harbors
Authorization: Bearer <token>
```

```
{
  "harbors": [
    {
      "harbor_name": "Split",
      "harbor_code": "HR479",
      "harbor_longitude": null,
      "harbor_latitude": null,
      "harbor_region": "LUČKA UPRAVA SPLIT",
      "harbor_country": null
    }
  ]
}
```

In every further call a harbor is identified by `harbor_code` .

2. Departure search

```
POST /search_trip
Authorization: Bearer <token>
```

```
{
  "travel_from": "HR479",
  "travel_to": "HR364",
  "travel_date": "2026-09-05",
  "control_code": "<sha512>"
}
```

```
{
  "trips": [
    {
      "trip_uuid": "9d01110a-2c95-44d4-b301-784708d9ff94",
      "departure": "05.09.2026. 08:00",
      "arrival": "05.09.2026. 09:00",
    }
  ]
}
```

```

    "departure_harbor_id": "HR479",
    "departure_harbor_name": "Split",
    "arrival_harbor_id": "HR364",
    "arrival_harbor_name": "Hvar",
    "line_code": "647",
    "line_name": "Split - Milna - Hvar - Korčula - Pomena - Dubrovnik",
    "prices": [
      {
        "ticket_type_uuid": "f6a99f5f-174d-4f00-a38b-f3e79dfc91a0",
        "ticket_type_name": "Redovna",
        "price": 25,
        "capacity": 100,
        "description": null
      }
    ]
  }
]
}

```

- `trip_uuid` is what you send in the order.
- **The price is your purchase price**, the one we invoice you at — not the price you charge the passenger. Depending on the agreement it is sent with or without VAT; the harbor fee is always included.
- Departure times are in local time, formatted `DD.MM.YYYY. HH:mm`.
- **Departures whose time has passed are not returned.** An empty `trips` means there is no departure on that route that day — it is not an error.

3. Order

```

POST /order
Authorization: Bearer <token>

{
  "order_number": "BW-2026-000123",
  "order_items": [
    {
      "trip_uuid": "9d01110a-2c95-44d4-b301-784708d9ff94",
      "ticket_type_uuid": "f6a99f5f-174d-4f00-a38b-f3e79dfc91a0",
      "ticket_type_name": "Redovna",
      "quantity": 2,
      "single_item_price": 25,
      "total_item_price": 50
    }
  ],

```

```
"control_code": "<sha512>"
}
```

```
{
  "msg": "order created",
  "order_uuid": "f0a1...",
  "order_number": "BW-2026-000123",
  "order_items": [ ... ]
}
```

`order_number` is **your** order number; it identifies the order later on the settlement. `order_uuid` is ours — store it, you need it for confirmation and cancellation.

If there are not enough free seats, the response is `409` and no order is created.

4. Order confirmation

```
POST /confirm_order
Authorization: Bearer <token>

{ "order_uuid": "f0a1...", "control_code": "<sha512>" }
```

```
{
  "tickets": [
    {
      "ticket_uuid": "...",
      "ticket_code": "HEXNYZ7DHA",
      "order_uuid": "f0a1...",
      "order_number": "BW-2026-000123",
      "ticket_type_name": "Redovna",
      "ticket_single_price": 25,
      "ticket_is_active": true,
      "ticket_is_canceled": false,
      "ticket_departure": "05.09.2026. 08:00",
      "line_code": "647",
      "ticket_departure_harbor_name": "Split",
      "ticket_arrival_harbor_name": "Hvar"
    }
  ]
}
```

Only now is the ticket issued. `ticket_code` is what the passenger presents at boarding — print it on the ticket and as a barcode or QR code.

Confirm only after payment has gone through on your side: from that moment the tickets exist and are charged, and can be undone only by cancellation.

5. Cancellation

```
POST /cancel_order
Authorization: Bearer <token>

{ "order_uuid": "f0a1...", "control_code": "<sha512>" }
```

```
{
  "msg": "tickets canceled",
  "return_amount": 50,
  "canceled_tickets": [ ... ]
}
```

Without additional fields the whole order is cancelled. To cancel part of it, send `tickets` with their `ticket_uuid`; `return_amount` is the amount credited to you.

A ticket that is already cancelled is not cancelled again.

6. Trip details

```
POST /trip_details
Authorization: Bearer <token>

{ "trip_uuid": "9d01110a-2c95-44d4-b301-784708d9ff94" }
```

```
{
  "trip_details": [
    {
      "departure_harbor_id": "HR479",
      "departure_harbor_name": "Split",
      "departure_planned": "05.09.2026. 08:00",
      "departure": "05.09.2026. 08:00",
      "arrival_harbor_id": "HR364",
      "arrival_harbor_name": "Hvar",
      "arrival_planned": "05.09.2026. 09:00",
      "arrival": "05.09.2026. 09:00",
      "harbor_order": 10
    }
  ]
}
```

Returns every harbor on that sailing, in sailing order — for showing intermediate stops and times. No control code required.

Errors

Status	When	Body
400	a field is missing or the control code does not match	<code>{"msg": "Invalid control_code"}</code>
401	no token, expired token, or wrong TID/OTP	<code>{"msg": "Unauthorized"}</code>
404	order or departure does not exist, or is not yours	<code>{"msg": "Order not found"}</code>
409	not enough free seats	<code>{"msg": "..."} </code>
429	too many requests per minute	—

Another partner's order returns `404` , not `403` — the response must not reveal whether it exists at all.

Billing

Tickets sold through the API are not paid for one by one. Sales are accumulated and invoiced **in bulk, on the agreed schedule**, at the prices you receive in `search_trip` . The settlement shows your `order_number` and the TID of the terminal the ticket was issued from.

Checklist before going live

- ☐ the key `k` stays on your server and is never sent in a request
 - ☐ the token is cached for 24 hours, not fetched on every call
 - ☐ control code fields follow exactly the prescribed order
 - ☐ `order_uuid` from the response is stored with the order
 - ☐ confirmation happens only after successful payment on your side
 - ☐ unconfirmed orders are not left open — they hold seats
 - ☐ `ticket_code` is on the ticket and machine-readable
 - ☐ an empty `trips` is shown as "no departure", not as an error
-

For credentials, price list changes, or moving to the production address, contact support.